

Milestone 4 – Track 1: HiCache (Hierarchical KV Cache)

CS349D, Spring 2026 – Hiva Mohammadzadeh

The milestone-3 engine evicted radix-cache leaves by **dropping** them: their KV pages went back to the GPU free list and the cached prefix was gone. On deep multi-turn / RAG-style workloads, once the working set crosses HBM capacity the next request that should have hit the cache has to re-prefill from scratch – and re-prefilling a long shared prefix on an 8B model is 50–150 ms, dominating TTFT under load.

HiCache replaces “drop” with **demote-to-CPU**: evicted GPU pages get copied into a pinned host-memory tier, the radix node stays in the tree marked `tier="cpu"`, and a later match against that prefix triggers a **CPU->GPU promote** before the request reads it. The CPU pool is bounded and runs its own LRU; only when both tiers can't make room does HiCache fall back to the m3 drop path (so eviction always makes progress).

Both the **full-credit bar** (blocking demote/promote with the right correctness story) and the **--hichache-overlap bonus** (dedicated CUDA stream + pinned memory + event-gated reuse) ship and run end-to-end on the L4. I'll be candid below about which parts of the cliff/restore demo the shipped `bench_cache.multiturn` workload exposes cleanly and which require either a re-access pattern the bench doesn't ship or an `nsys` capture I didn't quite land before the deadline.

1. Design and implementation

1.1 One radix tree, two tiers

The core idea is that the radix tree is **tier-aware** rather than GPU-only. Each `RadixNode` carries a single new field

```
class RadixNode:
    __slots__ = ("parent", "children", "key", "pages",
                "ref_count", "last_access", "tier")
    # tier in {"gpu", "cpu"}; node.pages is interpreted in that pool's
    # index space. Single-tier per node, no mixing.
```

and `RadixCache` gains an optional `cpu_pool` reference. **When `cpu_pool` is `None` (i.e. `--cpu-cache-size-gb 0`), every new code path is short-circuited and behavior is byte-identical to milestone 3** – the existing m3 test suite stays green unchanged, and the same binary serves as the baseline comparison.

`CpuKvPool` is a pinned-host mirror of `KVMemoryPool` with the same per-layer K/V tensor layout (`(num_pages, page_size, num_kv_heads, head_dim)`), so demote and promote are straight indexed copies, no reshaping:

```
# Demote (D2H), per layer
cpu.k_buffers[layer][cpu_slots] = gpu.k[layer][gpu_pages].to("cpu")
cpu.v_buffers[layer][cpu_slots] = gpu.v[layer][gpu_pages].to("cpu")
# Promote (H2D) is the symmetric reverse.
```

Pinned memory (`pin_memory=True`) is what lets these copies run as true async DMAs under `--hichache-overlap`.

1.2 Demote on eviction (the core hot path)

The change to `RadixCache.evict` is small but load-bearing. Pseudocode of the inner loop (real code in `miniengine/radix_cache.py`):

```
for node in lru_walk_gpu_tier_leaves(n_pages_needed):
    if not _try_demote(node):
        # CPU tier also full + uncooperative -> fall back to m3 drop.
        pool.free(node.pages)
```

```

        remove_from_tree(node)

def _try_demote(node):
    need = len(node.pages)
    if cpu_pool.num_free < need:
        _cpu_evict(need - cpu_pool.num_free)    # CPU-tier LRU
    if cpu_pool.num_free < need:
        return False                            # caller will drop
    cpu_slots = cpu_pool.allocate(need)
    copy_KV_D2H(node.pages, cpu_slots)         # async on copy_stream
                                              # if --hcache-overlap
                                              # or deferred_free in async mode

    pool.free(node.pages)
    node.tier = "cpu"
    node.pages = cpu_slots                     # NODE STAYS IN THE TREE

```

The critical line is the last comment: a demoted node **stays in the radix tree**. The prefix is still cached, just colder. A future `match_prefix` will walk right through it.

1.3 Promote on hit

`match_prefix` is read-only and tier-agnostic – it returns a path that may include CPU-tier nodes. The engine then calls a new `RadixCache.promote_match(match)` before locking the matched node:

```

def promote_match(match):
    if cpu_pool is None: return
    path = root_to_leaf(match.last_node)
    cpu_nodes = [n for n in path if n.tier == "cpu"]
    if not cpu_nodes: return                # all-GPU fast path

    inc_lock_ref(match.last_node)           # temp-lock pins the whole
    try:                                    # ancestor chain against
        for node in cpu_nodes:              # eviction during promote
            gpu_pages = pool.allocate(len(node.pages)) # may evict others
            copy_KV_H2D(node.pages, gpu_pages)
            cpu_pool.free(node.pages)       # or deferred_free under
            node.tier = "gpu"; node.pages = gpu_pages # --hcache-overlap
    finally:
        dec_lock_ref(match.last_node)
        rebuild_matched_pages(match, path)  # refresh to GPU indices

```

Two non-obvious bits:

- The **temp-lock** during promotion is essential. While we're allocating GPU pages for one node, that allocation may itself fire evict, which could otherwise pick a sibling on the same path and demote it back out from under us.
- `match.matched_pages` is rebuilt at the end so the engine downstream sees only GPU page indices in the request's page table.

Once promotion is done the engine `inc_lock_refs` the matched node again (separately, for the lifetime of the borrowing request) and proceeds exactly as in m3.

1.4 Async overlap (--hcache-overlap)

Off by default; the blocking path lands first. Under the flag:

- A dedicated `torch.cuda.Stream` is constructed at engine startup and threaded through to `RadixCache`.

- Demote and promote issue under `torch.cuda.stream(copy_stream)` with `non_blocking=True`. Pinned host memory is what makes these copies true async DMAs rather than synchronous bounce-buffer copies.
- A **pending-free queue** on each pool (`deferred_free(pages, event) + _drain_pending_free()`) keeps source pages reserved until their recording `cuda.Event` has fired. Otherwise a fresh `allocate` could hand out a page that the copy stream is still reading and corrupt the in-flight DMA. As a last resort, `allocate` blocks on the oldest pending event so the caller never sees a spurious OOM when capacity is genuinely available – just not yet released.
- On promote, the compute stream issues `current_stream.wait_event(promote_event)` – non-blocking on the CPU, serializes only the GPU stream – so flash-attn never reads half-copied KV.

1.5 Two pre-existing m3 bugs the rollout surfaced

Both in `miniengine/engine.py`, both double-frees:

1. `free_paged_request` inserted the finished request's full prompt+output into the radix cache and freed pages returned as redundant. But `req.page_table[:n_matched] == matched_node.pages` by construction – the “redundant” indices are physically the same pages the tree still references. So the pool got those indices back while the tree also kept them; when the tree later evicted the node it freed the same indices a second time.
2. `_insert_prompt_into_cache` (called after every prefill batch and chunk) had the identical shape.

Symptom under sustained multi-turn load: `pool_num_free` quietly drifts above `num_pages` (I observed 117 free / 98 capacity / 57 cached = 174 phantom indices), two requests get handed the same page, KV silently corrupts, eventually the scheduler wedges. The first on-pass demo I tried (32x5x256 conc=4) hung at request #158 from this. The fix is one line each – let the tree retain ownership of redundant indices. Commits `da070df`, `819eb13`. 59/59 tests pass before and after.

1.6 Module surface

File	Role
<code>miniengine/cpu_kv_pool.py</code> (<i>new</i>)	Pinned-host mirror of <code>KVMemoryPool</code> ; <code>from_budget</code> sizes from <code>--cpu-cache-size-gb</code> ; <code>deferred_free/_drain_pending_free</code> for the async path
<code>miniengine/radix_cache.py</code>	<code>RadixNode.tier</code> ; <code>RadixCache</code> accepts <code>cpu_pool/copy_stream/overlap</code> ; evict demotes; <code>new_cpu_evict/_promote_node/promote_match</code> ; <code>split</code> inherits child tier; new counters (<code>total_demoted_pages</code> , <code>total_promoted_pages</code> , ...)
<code>miniengine/engine.py</code>	Builds <code>CpuKvPool</code> when flag set; creates copy stream when <code>--hcache-overlap</code> ; calls <code>promote_match</code> in <code>_setup_paged_request</code> before lock
<code>miniengine/__main__.py</code>	<code>--cpu-cache-size-gb</code> <code>FLOAT</code> (default 0), <code>--hcache-overlap</code> , fail-fast validation
<code>miniengine/server.py</code>	<code>/cache_stats</code> grows a hcache subtree (CPU pool occupancy, demote/promote counters, copy-time accumulators)
<code>miniengine/kv_memory_pool.py</code>	<code>deferred_free/_drain_pending_free</code> (mirrors <code>CpuKvPool</code>); <code>allocate</code> syncs on oldest pending as a last resort

2. Performance

2.1 Setup

- **Hardware:** single NVIDIA L4 (23 GB HBM), 60 GB host RAM, no swap.
- **Model:** Qwen/Qwen3-8B in float16. Weights ~16 GB; at `--mem-fraction-static 0.85` the KV pool gets the remaining ~3.73 GB.
- **Engine config:** `--mode paged --page-size 256 --prefill-chunk-size 512`. The milestone example uses `--page-size 32`, which crashes at first prefill on L4 with *“Paged KV cache block size must be divisible by 256”* – a flash-attn 2.x constraint on Ada. (Same gotcha as m3.)
- **GPU KV pool:** 98 pages x 256 tokens = 25 088 cacheable tokens.
- **CPU KV tier:** `**--cpu-cache-size-gb 24 -> 635 slots x 256 tokens` 162 000 tokens, 6.5x the GPU pool**, pinned. (I tried 40 GB first; the L4 OOM-killed the server during boot. With 60 GB host RAM and no swap, 40 GB pinned + weights staging + Python tipped over. 24 GB lands safely under the ceiling.)

2.2 Correctness: MMLU is unchanged

`bench_accuracy --dataset mmlu --num-samples 200` against the same binary, once with HiCache off, once with HiCache on:

	HiCache OFF (m3)	HiCache ON
MMLU accuracy	61.5 % (123/200)	61.5 % (123/200)
Avg per-request latency	1.77 s	1.75 s

Identical to the integer. HiCache is a transparent cache: demote and promote are bitwise indexed copies, so KV is preserved exactly; there is no place along the path where output can drift. This is the cheapest end-to-end correctness evidence in the report.

2.3 Full-credit cache demo – 128-session multiturn

`bench_cache.py --workload multiturn --num-sessions 128 --turns-per-session 6 --max-tokens 192 --concurrency 32`, run twice with the same binary. 768 requests per pass.

Totals:

Metric	OFF (m3 baseline)	ON (HiCache, blocking)
Wall time	205.1 s	202.6 s
Throughput	3.74 req/s	3.79 req/s
Generation rate	163 tok/s	159 tok/s
Overall hit rate	32.7 %	32.0 %
Pages inserted	291	278
GPU pages dropped	192	0
Demoted GPU->CPU	–	200
Promoted CPU->GPU	–	7
Final num_cached_pages	99	85 (GPU) + 193 (CPU)

Per-turn:

turn	OFF hit	OFF TTFT_p50	ON hit	ON TTFT_p50
0	0.0%	491 ms	0.0%	476 ms
1	0.0%	521 ms	0.0%	508 ms
2	0.0%	405 ms	0.0%	496 ms
3	25.7%	452 ms	18.0%	477 ms
4	56.3%	355 ms	60.7%	319 ms
5	61.8%	324 ms	61.2%	358 ms

2.4 Async overlap smoke (--hichache-overlap ON)

384-request multiturn (64x6x192, conc=32, --hichache-overlap):

		Overlap ON
	Wall time	108.7 s
	Throughput	3.53 req/s
	Hit rate	31.3 %
	Pages demoted	54
total_demote_time_ms (issuer-side)	552 ms (avg ~10 ms / demote)	
	GPU evictions	0
	Pool integrity	cached+free = 91+7 = 98 = capacity OK

No errors, no corruption, no deadlocks under the conc=32 load that originally revealed the m3 double-free. The dedicated stream is logged at startup (HiCache overlap stream: <torch.cuda.Stream device=cuda:0 cuda_stream=. . .>) and the deferred-free queue does its job (pool accounting stays consistent).

2.5 Analysis: hardware and traffic effects

Three observations from the numbers:

(a) HiCache eliminated every GPU drop on this workload. Off-pass dropped 192 pages over the run; on-pass dropped zero and absorbed all 200 candidates into the CPU tier. That's the structural promise of the tier holding up under real load.

(b) Throughput and hit rate were essentially unchanged. This is where the bench_cache workload structure matters more than the cache implementation. Vanilla bench_cache.multiturn's worker pool pulls a session off the queue, runs **all** that session's turns sequentially, then picks the next session. So a session's prior-turn prefix is the **most recently** cached thing right when its next turn looks for it – it's never the LRU victim. Eviction (whether drop or demote) hits prefixes belonging to **finished** sessions, which the workload won't look at again. So the per-turn hit rate climbs identically in both passes and HiCache's avoided-re-prefill savings simply don't trigger.

(c) HiCache still promoted 7 prefixes back from CPU. Even on this workload, 7 re-access events occurred and HiCache responded correctly, restoring those prefixes from host memory in O(PCIe-bandwidth) time instead of forcing a re-prefill on the 8B target. The avoided- re-prefill cost for those 7 events is real (~1–2 ms H2D vs ~50–150 ms re-prefill on this hardware), but it's a rounding error against the 200 s total run.

(d) The L4's PCIe gen4 + 24 GB pinned tier is more than enough. The issuer-side demote time averages ~10 ms per demote at page_size=256, 36 layers (~36 MB per page across K and V). That's well below the ~50–150 ms re-prefill cost the demote replaces, so the structural break-even is firmly on HiCache's side – once the workload actually re-accesses, the win is large.

2.6 Pre-existing bug fixes were necessary to even get this far

Without the §1.5 double-free fixes, the on-pass demo I ran at conc=4 wedged at request #158: phantom free-list entries caused page reuse collisions and the scheduler couldn't drain the waiting queue. So part of the

milestone-4 deliverable was actually shoring up the m3 baseline to handle the load HiCache was designed for. All 59 tests pass and the 768-request smoke runs cleanly post-fix.

3. Next steps

3.1 Identified bottlenecks

The shipped workload doesn't reward HiCache. This is the biggest "gotcha" of the milestone for me. The spec describes a per-turn hit-rate **cliff** ($\geq 70\%$ turn 1, $< 20\%$ last turn), but vanilla `bench_cache.multiturn`'s session-at-a-time access pattern guarantees the LRU victim is the **finished** session's prefix, not anything the workload will re-access. A workload that interleaves session turns (round-robin) so older sessions are revisited after many newer ones have flushed the cache would expose the cliff cleanly; the small bench modification needed (swap the worker queue's enqueue order from session-by-session to turn-by-turn across sessions) is the obvious next step.

MMLU latency variance is high under HiCache. Per-request latency ranges 0.2 s to 4.3 s with mean 1.75 s; that's an artifact of `concurrency=1` + variable prompt size, not HiCache itself, but it emphasizes that the L4 with `--page-size 256` and 98 pages is genuinely tight – a single moderately-long prompt eats a meaningful fraction of HBM. A larger CPU tier or smaller page size (impossible on Ada with flash-attn 2.x) would help with longer-prompt workloads.

The ~10 ms issuer-side demote time is not yet hidden. In overlap mode the COPY runs on the dedicated stream and overlaps with concurrent compute on the default stream, but the **issuer-side Python wall time** to set up the copy (slice the GPU tensor, queue the H2D, record the event) still serializes against the engine loop. SGLang's HiCache batches these per-eviction-pass; we issue one event per demoted node. For a heavier eviction storm, batching event recording across all demotes in a single `evict` call would cut Python overhead.

3.2 Additional techniques implemented beyond batching

Beyond regular continuous batching (milestone 1) and paged KV + flash-attn varlen (milestone 2), this milestone landed:

1. **CPU-tier KV cache with bounded LRU.** The mechanism described in §1; this is the milestone's main deliverable. Capacity is sized in GB so the operator picks the tradeoff against host RAM directly.
2. **Dedicated CUDA stream + pinned-memory async H2D/D2H.** The `--hichache-overlap` path overlaps copy traffic with model compute, gated by `cuda.Event` so the compute stream never reads half-copied KV.
3. **Deferred-free with on-allocate drain + sync-as-last-resort.** Both pools hide async-in-flight pages from the free list until the recording event has fired. `allocate` syncs on the oldest pending event before raising `KVOutOfMemory`, so callers never see a spurious shortage when capacity is genuinely available.
4. **/cache_stats instrumentation.** Demote/promote counters, copy-time accumulators, CPU pool occupancy. Sufficient to verify HiCache is active and quantify its effect at runtime.
5. **Fixed two pre-existing m3 double-frees** that didn't matter at low concurrency but corrupted the pool's free list under sustained multi-turn load.

3.3 What would unlock the perf-win bonus

The ~20 % throughput / TTFT improvement bonus didn't land on the workloads I ran. The structural argument from §2.5 is sound – a single promotion costs $O(\text{milliseconds})$, a single re-prefill costs $O(\text{hundreds of milliseconds})$, so each avoided re-prefill is a ~50–100x win on that request's TTFT – but it requires **re-access** of demoted prefixes to materialize. Two paths forward I would take with another day:

- **Round-robin multiturn bench.** Modify `bench_cache.multiturn` to interleave sessions turn-by-turn rather than session-by-session. With `conc=32` and ~64 sessions this would force the LRU to evict prefixes that the workload then re-accesses, exactly the regime HiCache wins on.

- **nsys capture of the async path.** With `--hcache-overlap` on under heavy promote pressure, capturing a timeline and rendering a promote-time-hidden ratio is the cleanest evidence the overlap is real – the infrastructure (events, deferred-free, stream wait_event) is already in place and tested.

3.4 Status summary

Deliverable	Status
<code>--cpu-cache-size-gb</code> flag, byte-identical to m3 at 0	Done
GPU eviction -> CPU demote (blocking)	Done
Promote-on-hit with temp-lock + matched_pages rebuild	Done
CPU-tier LRU + fall-back drop	Done
<code>--hcache-overlap</code> (dedicated stream + pinned + deferred-free)	Done – boots, runs, pool integrity preserved
<code>/cache_stats</code> HiCache counters	Done
Unit tests (59/59, incl. bitwise round-trip)	Done
768-request L4 smoke	Done – no errors
MMLU accuracy unchanged (61.5 % == 61.5 %)	Done
Demote/promote mechanism verified end-to-end	Done – 200 demotes, 7 promotes, 0 drops
Per-turn cliff/restore on default <code>bench_cache.multiturn</code>	Partial – 192 GPU drops are eliminated, but per-turn average doesn't cliff because the workload's access pattern aligns with LRU
<code>nsys</code> timeline + promote-time-hidden ratio	Not captured
<code>>=20 %</code> throughput/TTFT win	Not achieved on this workload

The HiCache surface is built, tested, integrated, and runs cleanly at production scale on the L4. The bonuses that did not fully materialize needed either a workload variation with explicit prefix re-access (perf-win) or an `nsys` capture (overlap evidence) that I couldn't quite land in the remaining time.